

```

# Imports
from .ui import GameUI
from .board import ChessBoard
from .pieces import Piece
from src.ai.model import RandoBot,
ValueBot, SelfAwareBot, FutureSightBot
from src.ai.stopium import Stopium
import pygame, random, time

# Sentinel meaning "use the live game
state". None can't be the default here
# because None is a real en passant value -
it means "nothing is capturable"
USE_LIVE_STATE = object()

# Ray directions and piece types that
attack along them, shared by the
# fast attack-scanner and the check/pin
detector
ORTHOGONAL_DIRS = [(-1, 0), (1, 0), (0,
-1), (0, 1)]
DIAGONAL_DIRS = [(-1, -1), (-1, 1), (1,
-1), (1, 1)]
KNIGHT_OFFSETS = [(-2, -1), (-2, 1), (-1,
-2), (-1, 2), (1, -2), (1, 2), (2, -1), (2,
1)]

# Chess
class Chess():
    def __init__(self, ui:GameUI, mode:str,

```

```

colour:str):
    # UI
    self.ui = ui

    # Initialize game values
    self.player_turn = 'white'
    self.checkmate = False
    self.stalemate = False
    self.check = False
    self.move_history = []
    self.mode = mode
    self.player_colour = colour

    # TEMP
    if self.mode == 'bot':
        self.bot = Stopium()

    # Set player colour
    if self.player_colour is None:
        self.player_colour =
random.choice(['white', 'black'])

    # En passant tracking
    self.en_passant_square = None

    # Castling rights
    self.castling_rights = {
        'white': {'kingside': True,
'queenside': True},
        'black': {'kingside': True,
'queenside': True}
    }

    # create board

```

```

        self.chessboard =
ChessBoard().initialize_board(self.player_c
olour)

self.ui.update_screen(board=self.chessboard
)

        pygame.display.update()

        # start mode
        self.mainloop()

    def mainloop(self):
        while not self.checkmate and not
self.stalemate:
            is_valid_move = False
            while not is_valid_move:
                # Check for checkmate or
stalemate

                if self.is_game_over():
                    quit()

                # check who is next to move
                if self.player_colour ==
self.player_turn:

                    # Human move
                    legal_moves =
self.all_possible_moves(colour=self.player_
turn, board=self.chessboard)

                    sourceSquare,
targetSquare =
self.ui.run(board=self.chessboard,
turn=self.player_turn, game=self)

                    if sourceSquare and
targetSquare:

```

```

        is_valid_move =
self.validate_move(sourceSquare,
targetSquare)

        else:
            match self.mode:
                case 'local':
                    # Human move
                    legal_moves =
self.all_possible_moves(colour=self.player_
turn, board=self.chessboard)
                    sourceSquare,
targetSquare =
self.ui.run(board=self.chessboard,
turn=self.player_turn, game=self)
                    if sourceSquare
and targetSquare:

is_valid_move =
self.validate_move(sourceSquare,
targetSquare)

                case 'multiplayer':
                    pass

                case 'bot':
                    # Bot move
                    legal_moves =
self.all_possible_moves(colour=self.player_
turn, board=self.chessboard)
                    sourceSquare,
targetSquare =
self.bot.generate_move(board=self.chessboar
d, possible_moves=legal_moves,

```

```

turn=self.player_turn, game=self)
                                if sourceSquare
and targetSquare:

is_valid_move =
self.validate_move(sourceSquare,
targetSquare)

                                # Save move in history

self.move_history.append((sourceSquare,
targetSquare))

                                # Change turn
                                self.player_turn = "black" if
self.player_turn == "white" else "white"

                                # Update interface

self.ui.update_screen(board=self.chessboard
)

                                pygame.display.update()

def square_to_coords(self, square):
    file = ord(square[0]) - ord('a')
    rank = int(square[1]) - 1
    return (file, rank)

def coords_to_square(self, file, rank):
    return chr(ord('a') + file) +
str(rank + 1)

def is_game_over(self):
    legal_moves =

```

```

self.all_possible_moves(self.player_turn,
self.chessboard)

        if len(legal_moves) == 0:
            if
self.is_in_check(self.player_turn,
self.chessboard):
                self.checkmate = True
                print(f"Checkmate!
{{'Black' if self.player_turn == 'white'
else 'White'}} wins!")
            else:
                self.stalemate = True
                print("Stalemate!")
                return True

        return False

    def all_possible_moves(self, colour,
board, en_passant=USE_LIVE_STATE,
castling=USE_LIVE_STATE):
        if en_passant is USE_LIVE_STATE:
            en_passant =
self.en_passant_square
        if castling is USE_LIVE_STATE:
            castling = self.castling_rights

        king_square =
self.find_king(colour, board)
        if king_square is None:
            return []

        # Work out what's checking the king
and which friendly pieces are

```

```

        # pinned ONCE for this position,
        instead of copying the board and
        # re-scanning for attacks after
        every single candidate move.
        checkers, pins =
self.get_checkers_and_pins(king_square,
colour, board)
        num_checkers = len(checkers)

        if num_checkers == 1:
            # A single check can only be
            resolved by capturing the checker
            # or (for a slider) blocking a
            square between it and the king.
            resolve_squares = checkers[0]
[1] | {checkers[0][0]}
        else:
            # 0 checkers: no restriction.
            2+ checkers: only the king can move.
            resolve_squares = None

        legal_moves = []

        for square, square_data in
board.items():
            piece_obj =
square_data['piece']
            if piece_obj is None or
piece_obj.colour != colour:
                continue

            is_king = piece_obj.piece_type
== 'King'
            if num_checkers >= 2 and not

```

```

is_king:
    continue

    pseudo_legal =
self.get_pseudo_legal_moves(square, board,
en_passant=en_passant, castling=castling)
    pin_ray = pins.get(square)

    for target in pseudo_legal:
        if is_king:
            # The king can't step
onto an attacked square. Exclude its
            # own origin square
from the scan, otherwise it looks like
            # it blocks a slider's
ray behind it (it's about to move).
            if not
self.is_square_attacked(target, colour,
board, en_passant, castling,
exclude_square=square):

legal_moves.append((square, target))
            continue

            # En passant can expose a
discovered check along the rank in
            # a way pin/check detection
above doesn't model (both pawns
            # leave the rank at once).
It's rare, so just verify it fully.
            if piece_obj.piece_type ==
'Pawn' and en_passant is not None and
target == en_passant:
                if

```



```

self.is_legal_move(square, target, board,
en_passant, castling):

    legal_moves.append((square, target))
        continue

        if pin_ray is not None and
target not in pin_ray:
            continue
        if resolve_squares is not
None and target not in resolve_squares:
            continue

        legal_moves.append((square,
target))

    return legal_moves

def get_checkers_and_pins(self,
king_square, colour, board):
    """Ray-cast out from the king once
to find every piece giving check
    and every friendly piece that's
absolutely pinned.

    Returns (checkers, pins):
    - checkers: list of
(attacker_square, blocking_squares_set)
pairs.
        blocking_squares_set is empty for
knight/pawn checks (can't be blocked).
    - pins: {pinned_piece_square:
allowed_squares_set}, where
allowed_squares_set

```

```

        is every square along the pin ray
the pinned piece may still move to
        (including capturing the pinning
piece).
"""
        attacker_colour = 'black' if colour
== 'white' else 'white'
        file0, rank0 =
self.square_to_coords(king_square)
        checkers = []
        pins = {}

        for directions, slider_types in
((ORTHOGONAL_DIRS, ('Rook', 'Queen')),
(DIAGONAL_DIRS, ('Bishop', 'Queen'))):
            for df, dr in directions:
                f, r = file0 + df, rank0 +
dr

                between = []
                blocker_square = None
                while 0 <= f < 8 and 0 <= r
< 8:
                    sq =
self.coords_to_square(f, r)
                    piece = board[sq]
['piece']

                    if piece is None:
                        between.append(sq)
                    elif blocker_square is
None and piece.colour == colour:
                        blocker_square = sq
                    elif piece.colour !=
colour and piece.piece_type in
slider_types:

```

```

        if blocker_square
is None:

checkers.append((sq, set(between)))
        else:

pins[blocker_square] = set(between) | {sq}
        break
        else:
            break
        f += df
        r += dr

    for df, dr in KNIGHT_OFFSETS:
        f, r = file0 + df, rank0 + dr
        if 0 <= f < 8 and 0 <= r < 8:
            piece =
board[self.coords_to_square(f, r)][ 'piece' ]
            if piece and piece.colour
== attacker_colour and piece.piece_type ==
'Knight':

checkers.append((self.coords_to_square(f,
r), set()))

        direction = 1 if attacker_colour ==
'white' else -1
        for df in (-1, 1):
            f, r = file0 + df, rank0 -
direction
            if 0 <= f < 8 and 0 <= r < 8:
                piece =
board[self.coords_to_square(f, r)][ 'piece' ]
                if piece and piece.colour

```

```

== attacker_colour and piece.piece_type ==
'Pawn':

checkers.append((self.coords_to_square(f,
r), set()))

        return checkers, pins

    def get_pseudo_legal_moves(self,
square, board, include_castling=True,
en_passant=USE_LIVE_STATE,
castling=USE_LIVE_STATE):
        piece_obj = board[square]['piece']
        if piece_obj is None:
            return []

        piece_type = piece_obj.piece_type
        colour = piece_obj.colour
        moves = []

        match piece_type:
            case 'Pawn':
                moves =
self.get_pawn_moves(square, colour, board,
en_passant)
            case 'Knight':
                moves =
self.get_knight_moves(square, colour,
board)
            case 'Bishop':
                moves =
self.get_bishop_moves(square, colour,
board)
            case 'Rook':

```

```

        moves =
self.get_rook_moves(square, colour, board)
        case 'Queen':
            moves =
self.get_queen_moves(square, colour, board)
        case 'King':
            moves =
self.get_king_moves(square, colour, board,
include_castling, en_passant, castling)

    return moves

    def get_pawn_moves(self, square,
colour, board, en_passant=USE_LIVE_STATE):
        if en_passant is USE_LIVE_STATE:
            en_passant =
self.en_passant_square

        file, rank =
self.square_to_coords(square)
        moves = []
        direction = 1 if colour == 'white'
else -1
        start_rank = 1 if colour == 'white'
else 6

        # Forward move
        new_rank = rank + direction
        if 0 <= new_rank < 8:
            target_square =
self.coords_to_square(file, new_rank)
            if board[target_square]
['piece'] is None:
                moves.append(target_square)

```

```

        # Double move from starting
position
        if rank == start_rank:
            new_rank2 = rank + 2 *
direction
            target_square2 =
self.coords_to_square(file, new_rank2)
            if
board[target_square2]['piece'] is None:

moves.append(target_square2)

        # Captures
        for df in [-1, 1]:
            new_file = file + df
            if 0 <= new_rank < 8 and 0 <=
new_file < 8:
                target_square =
self.coords_to_square(new_file, new_rank)
                target_piece =
board[target_square]['piece']
                if target_piece and
target_piece.colour != colour:

moves.append(target_square)

        # En passant
        if en_passant:
            ep_file, ep_rank =
self.square_to_coords(en_passant)
            if rank + direction == ep_rank
and abs(file - ep_file) == 1:
                moves.append(en_passant)

```

```

        return moves

    def get_knight_moves(self, square,
        colour, board):
        file, rank =
self.square_to_coords(square)
        moves = []
        knight_offsets = [(-2,-1), (-2,1),
        (-1,-2), (-1,2), (1,-2), (1,2), (2,-1),
        (2,1)]

        for df, dr in knight_offsets:
            new_file, new_rank = file + df,
rank + dr
            if 0 <= new_file < 8 and 0 <=
new_rank < 8:
                target_square =
self.coords_to_square(new_file, new_rank)
                target =
board[target_square]['piece']
                if target is None or
target.colour != colour:

moves.append(target_square)

        return moves

    def get_sliding_moves(self, square,
        colour, board, directions):
        file, rank =
self.square_to_coords(square)
        moves = []

```

```

        for df, dr in directions:
            new_file, new_rank = file + df,
rank + dr
            while 0 <= new_file < 8 and 0
<= new_rank < 8:
                target_square =
self.coords_to_square(new_file, new_rank)
                target =
board[target_square]['piece']
                if target is None:

moves.append(target_square)
                else:
                    if target.colour !=
colour:

moves.append(target_square)
                    break
                    new_file += df
                    new_rank += dr

        return moves

    def get_bishop_moves(self, square,
colour, board):
        return
self.get_sliding_moves(square, colour,
board, [(-1,-1), (-1,1), (1,-1), (1,1)])

    def get_rook_moves(self, square,
colour, board):
        return
self.get_sliding_moves(square, colour,
board, [(-1,0), (1,0), (0,-1), (0,1)])

```



```

        def get_queen_moves(self, square,
colour, board):
            return
self.get_sliding_moves(square, colour,
board,

[(-1,-1), (-1,1), (1,-1), (1,1), (-1,0),
(1,0), (0,-1), (0,1)])

        def get_king_moves(self, square,
colour, board, include_castling=True,
en_passant=USE_LIVE_STATE,
castling=USE_LIVE_STATE):
            if castling is USE_LIVE_STATE:
                castling = self.castling_rights

            file, rank =
self.square_to_coords(square)
            moves = []

            # Normal king moves (always
included)
            for df in [-1, 0, 1]:
                for dr in [-1, 0, 1]:
                    if df == 0 and dr == 0:
                        continue
                    new_file, new_rank = file +
df, rank + dr
                    if 0 <= new_file < 8 and 0
<= new_rank < 8:
                        target_square =
self.coords_to_square(new_file, new_rank)
                        target =

```

```

board[target_square]['piece']
            if target is None or
target.colour != colour:

moves.append(target_square)

        # Castling (only when generating
actual moves, not checking attacks)
        if include_castling and not
self.is_in_check(colour, board, en_passant,
castling):
            # Kingside castling
            if castling[colour]
['kingside']:
                if
self.can_castle_kingside(colour, board,
en_passant, castling):

moves.append(self.coords_to_square(file +
2, rank))

            # Queenside castling
            if castling[colour]
['queenside']:
                if
self.can_castle_queenside(colour, board,
en_passant, castling):

moves.append(self.coords_to_square(file -
2, rank))

        return moves

    def can_castle_kingside(self, colour,

```

```

board, en_passant=USE_LIVE_STATE,
castling=USE_LIVE_STATE):
    rank = 0 if colour == 'white' else
7
    # Check squares are empty
    f_square = self.coords_to_square(5,
rank)
    g_square = self.coords_to_square(6,
rank)
    if board[f_square]['piece'] is not
None or board[g_square]['piece'] is not
None:
        return False

    # Check king doesn't pass through
check
    for file in [4, 5, 6]:
        test_board =
self.copy_board_state(board)
        e_square =
self.coords_to_square(4, rank)
        target_square =
self.coords_to_square(file, rank)
        test_board[target_square]
['piece'] = board[e_square]['piece']
        test_board[e_square]['piece'] =
None
        if
self.is_square_attacked(target_square,
colour, test_board, en_passant, castling):
            return False

    return True

```

```

        def can_castle_queenside(self, colour,
board, en_passant=USE_LIVE_STATE,
castling=USE_LIVE_STATE):
            rank = 0 if colour == 'white' else
7

            # Check squares are empty
            b_square = self.coords_to_square(1,
rank)
            c_square = self.coords_to_square(2,
rank)
            d_square = self.coords_to_square(3,
rank)

            if board[b_square]['piece'] is not
None or board[c_square]['piece'] is not
None or board[d_square]['piece'] is not
None:

                return False

            # Check king doesn't pass through
check
            for file in [4, 3, 2]:
                test_board =
self.copy_board_state(board)
                e_square =
self.coords_to_square(4, rank)
                target_square =
self.coords_to_square(file, rank)
                test_board[target_square]
['piece'] = board[e_square]['piece']
                test_board[e_square]['piece'] =
None
                if

```

```

self.is_square_attacked(target_square,
colour, test_board, en_passant, castling):
    return False

    return True

def copy_board_state(self, board):
    board_copy = {}
    for square, square_data in
board.items():
        board_copy[square] = {
            'square':
square_data['square'],
            'piece':
square_data['piece']
        }
    return board_copy

def copy_castling_rights(self,
castling=USE_LIVE_STATE):
    if castling is USE_LIVE_STATE:
        castling = self.castling_rights
    return {colour: rights.copy() for
colour, rights in castling.items()}

# Piece objects are shared between
boards, so a promotion in a simulated
# line must never edit one - reuse a
cached piece per type and colour instead
_simulated_pieces = {}

def simulated_piece(self, name,
colour):
    piece =

```

```

self._simulated_pieces.get((name, colour))
    if piece is None:
        piece = Piece(name, colour)
        self._simulated_pieces[(name,
colour)] = piece
    return piece

    # Apply a move to a copy of the
position and hand back the whole resulting
    # state, so a bot can search replies
without touching the real game
    def simulate_move(self, sourceSquare,
targetSquare, board,
en_passant=USE_LIVE_STATE,
castling=USE_LIVE_STATE,
promote_to='Queen'):
        if en_passant is USE_LIVE_STATE:
            en_passant =
self.en_passant_square

            new_board =
self.copy_board_state(board)
            new_castling =
self.copy_castling_rights(castling)
            piece_obj = new_board[sourceSquare]
['piece']
            if piece_obj is None:
                return new_board, None,
new_castling

            colour = piece_obj.colour
            opponent = 'black' if colour ==
'white' else 'white'
            source_file, source_rank =

```

```

self.square_to_coords(sourceSquare)
    target_file, target_rank =
self.square_to_coords(targetSquare)

    # En passant capture removes the
pawn beside the target, not on it
    if piece_obj.piece_type == 'Pawn'
and targetSquare == en_passant:
        captured_pawn_square =
self.coords_to_square(target_file,
source_rank)
        new_board[captured_pawn_square]
['piece'] = None

    # Only a double pawn push leaves a
square capturable next move
    new_en_passant = None
    if piece_obj.piece_type == 'Pawn'
and abs(target_rank - source_rank) == 2:
        mid_rank = (source_rank +
target_rank) // 2
        new_en_passant =
self.coords_to_square(source_file,
mid_rank)

    # Castling moves the rook too, and
gives up both rights
    if piece_obj.piece_type == 'King':
        if abs(target_file -
source_file) == 2:
            if target_file >
source_file:
                rook_source =
self.coords_to_square(7, source_rank)

```

```

        rook_target =
self.coords_to_square(5, source_rank)
        else:
            rook_source =
self.coords_to_square(0, source_rank)
            rook_target =
self.coords_to_square(3, source_rank)
            new_board[rook_target]
['piece'] = new_board[rook_source]['piece']
            new_board[rook_source]
['piece'] = None

            new_castling[colour]
['kingside'] = False
            new_castling[colour]
['queenside'] = False

        # Moving a rook gives up that
side's rights
        if piece_obj.piece_type == 'Rook':
            if source_file == 0:
                new_castling[colour]
['queenside'] = False
            elif source_file == 7:
                new_castling[colour]
['kingside'] = False

        # Capturing a rook gives up the
opponent's rights on that side
        captured_piece =
new_board[targetSquare]['piece']
        if captured_piece and
captured_piece.piece_type == 'Rook':
            if target_file == 0:

```



```

        new_castling[opponent]
['queenside'] = False
        elif target_file == 7:
            new_castling[opponent]
['kingside'] = False

        # Make the move
        new_board[targetSquare]['piece'] =
piece_obj
        new_board[sourceSquare]['piece'] =
None

        # Promotion
        if piece_obj.piece_type == 'Pawn'
and target_rank in (0, 7):
            new_board[targetSquare]
['piece'] =
self.simulated_piece(promote_to, colour)

        return new_board, new_en_passant,
new_castling

    def piece_moves(self, square, board,
en_passant=USE_LIVE_STATE,
castling=USE_LIVE_STATE):
        if square is None:
            return []
        legal_moves = []
        moves =
self.get_pseudo_legal_moves(square, board,
en_passant=en_passant, castling=castling)
        for target in moves:
            if self.is_legal_move(square,
target, board, en_passant=en_passant,

```

```

castling=castling):
    legal_moves.append((square,
target))
    return legal_moves

    def is_legal_move(self, source, target,
board, en_passant=USE_LIVE_STATE,
castling=USE_LIVE_STATE):
        if en_passant is USE_LIVE_STATE:
            en_passant =
self.en_passant_square

            # Make the move on a copy of the
board
            test_board =
self.copy_board_state(board)
            piece_obj = test_board[source]
['piece']
            colour = piece_obj.colour

            # Handle en passant capture
            if piece_obj.piece_type == 'Pawn'
and target == en_passant:
                source_file, source_rank =
self.square_to_coords(source)
                target_file, target_rank =
self.square_to_coords(target)
                captured_pawn_square =
self.coords_to_square(target_file,
source_rank)

            test_board[captured_pawn_square]['piece'] =
None

```

```

        # Make the move
        test_board[target]['piece'] =
piece_obj
        test_board[source]['piece'] = None

        # Check if king is in check
        return not self.is_in_check(colour,
test_board, en_passant, castling)

    def is_in_check(self, colour, board,
en_passant=USE_LIVE_STATE,
castling=USE_LIVE_STATE):
        # Find king position
        king_square =
self.find_king(colour, board)
        if not king_square:
            return False

        return
self.is_square_attacked(king_square,
colour, board, en_passant, castling)

    def is_square_attacked(self, square,
defender_colour, board,
en_passant=USE_LIVE_STATE,
castling=USE_LIVE_STATE,
exclude_square=None):
        # Direct scan instead of generating
every enemy piece's full move
        # list just to check membership -
stops the instant an attacker is
        # found. exclude_square lets a
king-move check pretend the king's
        # own origin square is empty, since

```

```

it's the square being vacated.
        attacker_colour = 'black' if
defender_colour == 'white' else 'white'
        file0, rank0 =
self.square_to_coords(square)

        def piece_at(sq):
            return None if sq ==
exclude_square else board[sq]['piece']

        for directions, slider_types in
((ORTHOGONAL_DIRS, ('Rook', 'Queen')),
(DIAGONAL_DIRS, ('Bishop', 'Queen'))):
            for df, dr in directions:
                f, r = file0 + df, rank0 +
dr
                while 0 <= f < 8 and 0 <= r
< 8:
                    piece =
piece_at(self.coords_to_square(f, r))
                    if piece is not None:
                        if piece.colour ==
attacker_colour and piece.piece_type in
slider_types:
                            return True
                            break
                    f += df
                    r += dr

        for df, dr in KNIGHT_OFFSETS:
            f, r = file0 + df, rank0 + dr
            if 0 <= f < 8 and 0 <= r < 8:
                piece =
piece_at(self.coords_to_square(f, r))

```

```

        if piece and piece.colour
== attacker_colour and piece.piece_type ==
'Knight':

            return True

        for df in (-1, 0, 1):
            for dr in (-1, 0, 1):
                if df == 0 and dr == 0:
                    continue
                f, r = file0 + df, rank0 +
dr
                if 0 <= f < 8 and 0 <= r <
8:
                    piece =
piece_at(self.coords_to_square(f, r))
                    if piece and
piece.colour == attacker_colour and
piece.piece_type == 'King':
                        return True

            direction = 1 if attacker_colour ==
'white' else -1
            for df in (-1, 1):
                f, r = file0 + df, rank0 -
direction
                if 0 <= f < 8 and 0 <= r < 8:
                    piece =
piece_at(self.coords_to_square(f, r))
                    if piece and piece.colour
== attacker_colour and piece.piece_type ==
'Pawn':
                        return True

    return False

```

```

    def find_king(self, colour, board):
        for square, square_data in
board.items():
            piece = square_data['piece']
            if piece and piece.piece_type
== 'King' and piece.colour == colour:
                return square
        return None

    def check_pawn_promotion(self, piece,
targetSquare, promoteToValue=None):
        square =
self.chessboard[targetSquare]["square"]
        x_pos, y_pos = square.topleft

        if promoteToValue is None:
            # If pawn is near the top of
the screen, menu goes DOWN
            # If pawn is near the bottom,
menu goes UP
            if y_pos < 320:
                rect = pygame.Rect(x_pos,
y_pos, 80, 80 * 4)
                reverse_images = False
            else:
                rect = pygame.Rect(x_pos,
y_pos - 80 * 3, 80, 80 * 4)
                reverse_images = True

        # Draw promotion area

self.ui.update_screen(board=self.chessboard
)

```

```

pygame.draw.rect(self.ui.Screen, (255, 255,
255), rect)

pygame.draw.rect(self.ui.Screen, (0, 0, 0),
rect, 2)

        # Draw the promotion choices
        rectangles =
self.pawn_promotion_ui(rect, piece.colour,
reverse_images)
        pygame.display.update()

        # Wait for player click
        piece_clicked = None
        while piece_clicked is None:
            for event in
pygame.event.get():
                if event.type ==
pygame.QUIT:
                    pygame.quit()
                    return None
                elif event.type ==
pygame.MOUSEBUTTONDOWN:
                    mouse_pos =
pygame.mouse.get_pos()
                    for rectangle in
rectangles:
                        if
rectangle.collidepoint(mouse_pos):
piece_clicked = rectangle
                                break
                    pygame.time.wait(10)

```

```

        promotion_map = {
            0: ("Queen", "q"),
            1: ("Knight", "n"),
            2: ("Bishop", "b"),
            3: ("Rook", "r"),
        }

        for i, rect_item in
enumerate(rectangles):
            if rect_item ==
piece_clicked:
                name, new_piece_type =
promotion_map[i]
                NEW_piece = Piece(name,
piece.colour)
                break

            else:
                promotion_map = {
                    "q": "Queen",
                    "n": "Knight",
                    "b": "Bishop",
                    "r": "Rook",
                }
                name =
promotion_map.get(promoteToValue.lower())
                if not name:
                    raise ValueError(f"Invalid
promotion value: {promoteToValue}")
                NEW_piece = Piece(name,
piece.colour)
                new_piece_type =
promoteToValue.lower()

```



```

        self.chessboard[targetSquare]
['piece'] = NEW_piece

self.ui.update_screen(board=self.chessboard
)

    pygame.display.update()
    return new_piece_type


def pawn_promotion_ui(self, rect,
colour, reverse_images=False):
    temp_list = []
    image_path = r"assets"
    piece_names = ["queen", "knight",
"bishop", "rook"]

    pieces = [
        pygame.image.load(f"
{image_path}/{name}_{colour}.png").convert_
alpha()
        for name in piece_names
    ]

    x_pos, y_pos = rect.topleft

    if reverse_images:
        positions = [rect.bottom - 80 *
(i + 1) for i in range(4)]
    else:
        positions = [rect.top + 80 * i
for i in range(4)]

    for i, piece_image in

```

```

enumerate(pieces):
    square_rect =
pygame.Rect(x_pos, positions[i], 80, 80)
    piece_rect =
piece_image.get_rect(center=square_rect.cen
ter)

self.ui.Screen.blit(piece_image,
piece_rect)
    temp_list.append(square_rect)

    return temp_list

    def validate_move(self, sourceSquare,
targetSquare):
        piece_obj =
self.chessboard[sourceSquare]['piece']

        # Check if there's a piece at
source
        if piece_obj is None:
            return False

        # Check if it's the right colour's
turn
        if piece_obj.colour !=
self.player_turn:
            return False

        # Check if the move is in the list
of legal moves
        legal_moves =
self.all_possible_moves(self.player_turn,
self.chessboard)

```

```

        if (sourceSquare, targetSquare) not
in legal_moves:
            return False

        # Handle en passant capture
        if piece_obj.piece_type == 'Pawn'
and targetSquare == self.en_passant_square:
            source_file, source_rank =
self.square_to_coords(sourceSquare)
            target_file, target_rank =
self.square_to_coords(targetSquare)
            captured_pawn_square =
self.coords_to_square(target_file,
source_rank)

self.chessboard[captured_pawn_square]
['piece'] = None

        # Update en passant square
self.en_passant_square = None
        if piece_obj.piece_type == 'Pawn':
            source_file, source_rank =
self.square_to_coords(sourceSquare)
            target_file, target_rank =
self.square_to_coords(targetSquare)
            if abs(target_rank -
source_rank) == 2:
                mid_rank = (source_rank +
target_rank) // 2
                self.en_passant_square =
self.coords_to_square(source_file,
mid_rank)

        # Handle castling

```

```

        if piece_obj.piece_type == 'King':
            source_file, source_rank =
self.square_to_coords(sourceSquare)
            target_file, target_rank =
self.square_to_coords(targetSquare)

            if abs(target_file -
source_file) == 2:
                # Kingside castling
                if target_file >
source_file:
                    rook_source =
self.coords_to_square(7, source_rank)
                    rook_target =
self.coords_to_square(5, source_rank)

self.chessboard[rook_target]['piece'] =
self.chessboard[rook_source]['piece']

self.chessboard[rook_source]['piece'] =
None

                # Queenside castling
                else:
                    rook_source =
self.coords_to_square(0, source_rank)
                    rook_target =
self.coords_to_square(3, source_rank)

self.chessboard[rook_target]['piece'] =
self.chessboard[rook_source]['piece']

self.chessboard[rook_source]['piece'] =
None

```

```

        # Update castling rights

self.castling_rights[self.player_turn]
['kingside'] = False

self.castling_rights[self.player_turn]
['queenside'] = False

        # Update castling rights if rook
moves or is captured
        if piece_obj.piece_type == 'Rook':
            source_file, source_rank =
self.square_to_coords(sourceSquare)
            if source_file == 0:

self.castling_rights[self.player_turn]
['queenside'] = False
                elif source_file == 7:

self.castling_rights[self.player_turn]
['kingside'] = False

        # Check if a rook is captured
(affects opponent's castling rights)
        captured_piece =
self.chessboard[targetSquare]['piece']
        if captured_piece and
captured_piece.piece_type == 'Rook':
            opponent = 'black' if
self.player_turn == 'white' else 'white'
            target_file, target_rank =
self.square_to_coords(targetSquare)
            if target_file == 0:

```

```

self.castling_rights[opponent]['queenside']
= False
        elif target_file == 7:

self.castling_rights[opponent]['kingside']
= False

        # Make the move
        self.chessboard[targetSquare]
['piece'] = piece_obj
        self.chessboard[sourceSquare]
['piece'] = None

        # Handle pawn promotion
        if piece_obj.piece_type == 'Pawn':
            target_file, target_rank =
self.square_to_coords(targetSquare)
            if (self.player_turn == 'white'
and target_rank == 7) or \
                (self.player_turn == 'black'
and target_rank == 0):
                if self.mode == 'bot' and
self.player_turn != self.player_colour:
                    # Ask the bot instead
of opening the UI, defaulting to a
                    # queen so a bot that
returns nothing can't hang the game
                    promote_to =
self.bot.choose_promotion(board=self.chessb
oard, targetSquare=targetSquare,
turn=self.player_turn, game=self) or 'q'

self.check_pawn_promotion(piece_obj,
targetSquare, promoteToValue=promote_to)

```

```
        else:
            # Call the pawn
            promotion UI

            self.check_pawn_promotion(piece_obj,
            targetSquare)

            # Update check status
            opponent = 'black' if
            self.player_turn == 'white' else 'white'
            self.check =
            self.is_in_check(opponent, self.chessboard)

            return True
```